

Software Engineering COMP1035

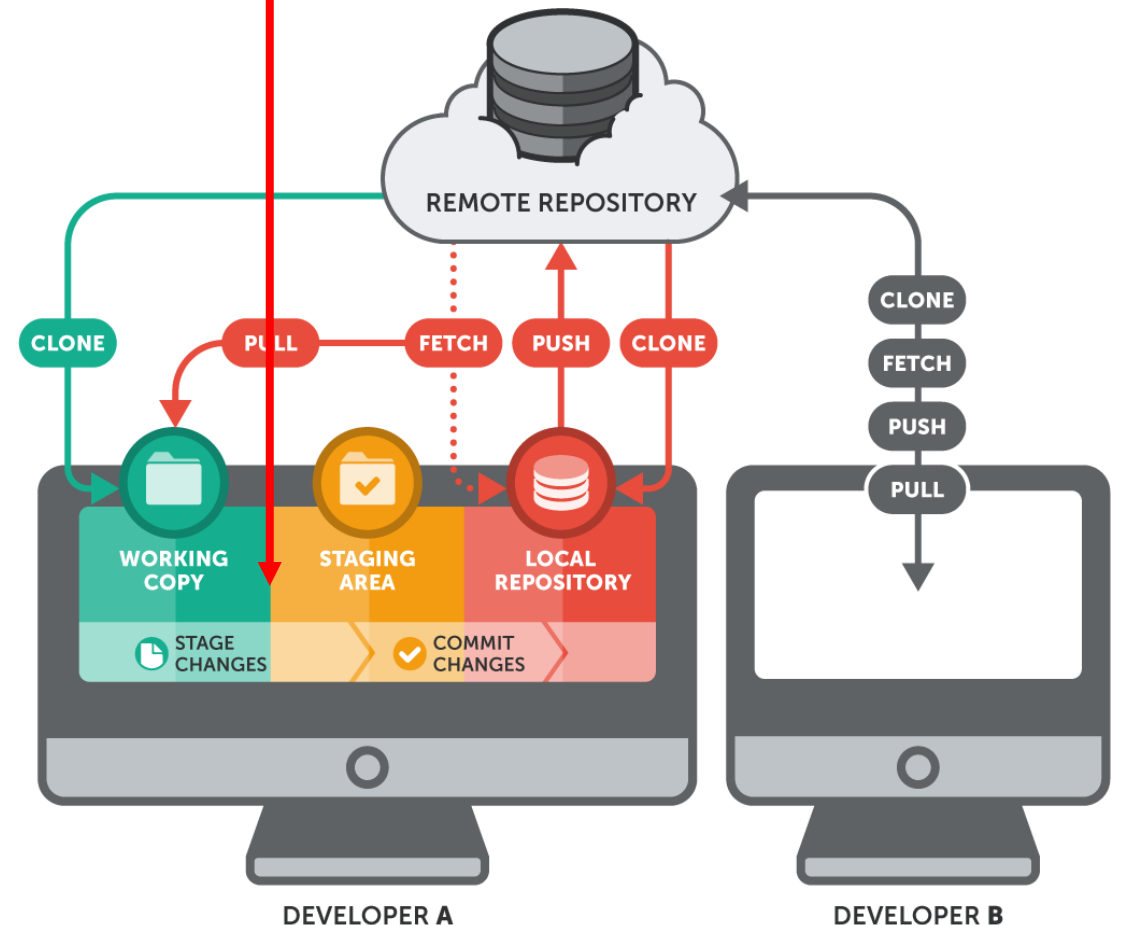
Lecture 09

Git Workflow and Advanced Git



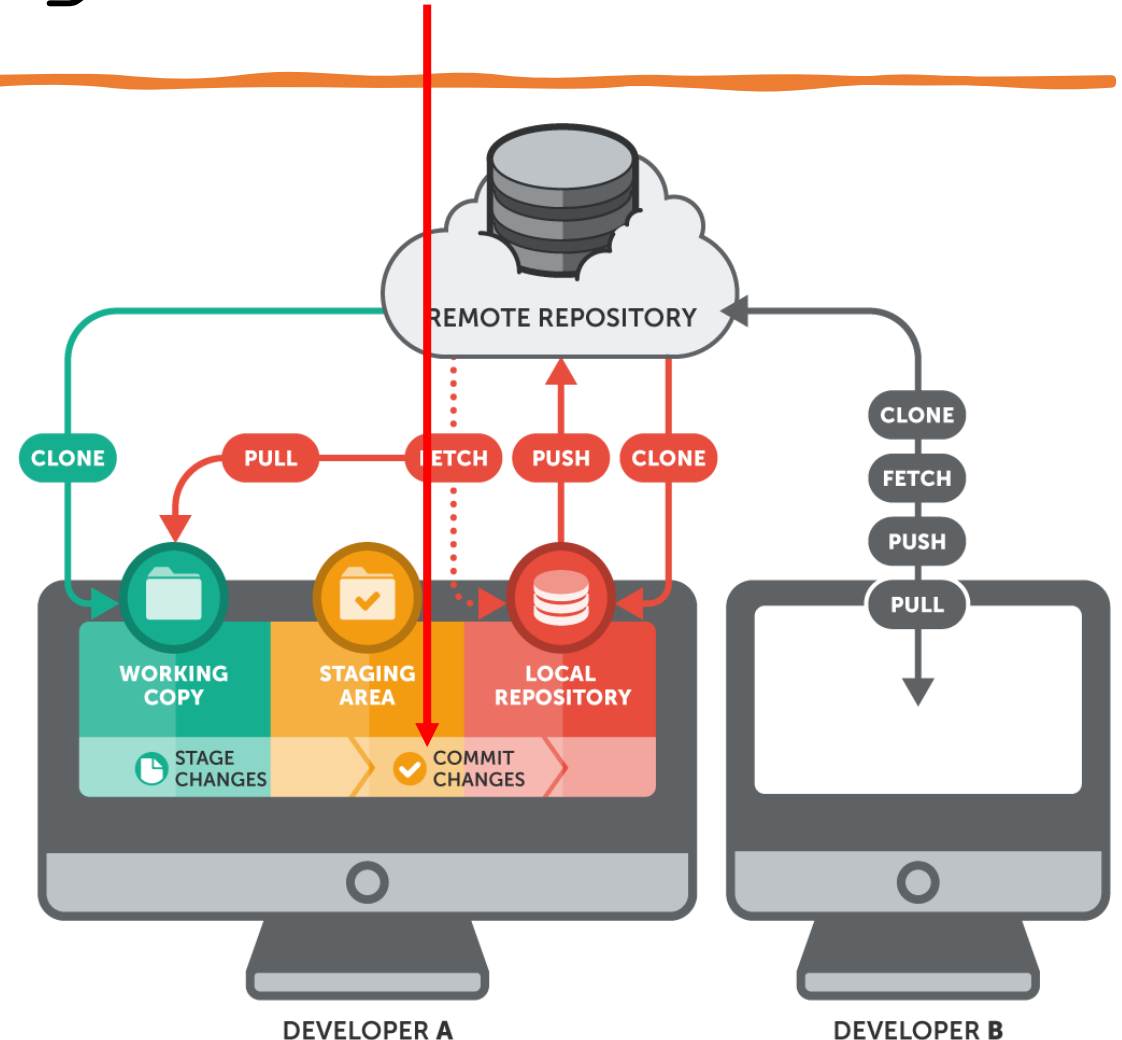
Recap of Basic Git - `git add`

- The `git add` command moves changes from your working directory to the staging area (also known as index).
- Staging area is a preparation area where you gather the changes you want to include in your next commit.
- Command usages:
 - `git add <filename>`: Adds a specific file.
 - `git add .`: Adds all changes in the current directory and its subdirectories.
 - `git add -A`: Adds all changes in the entire repository.



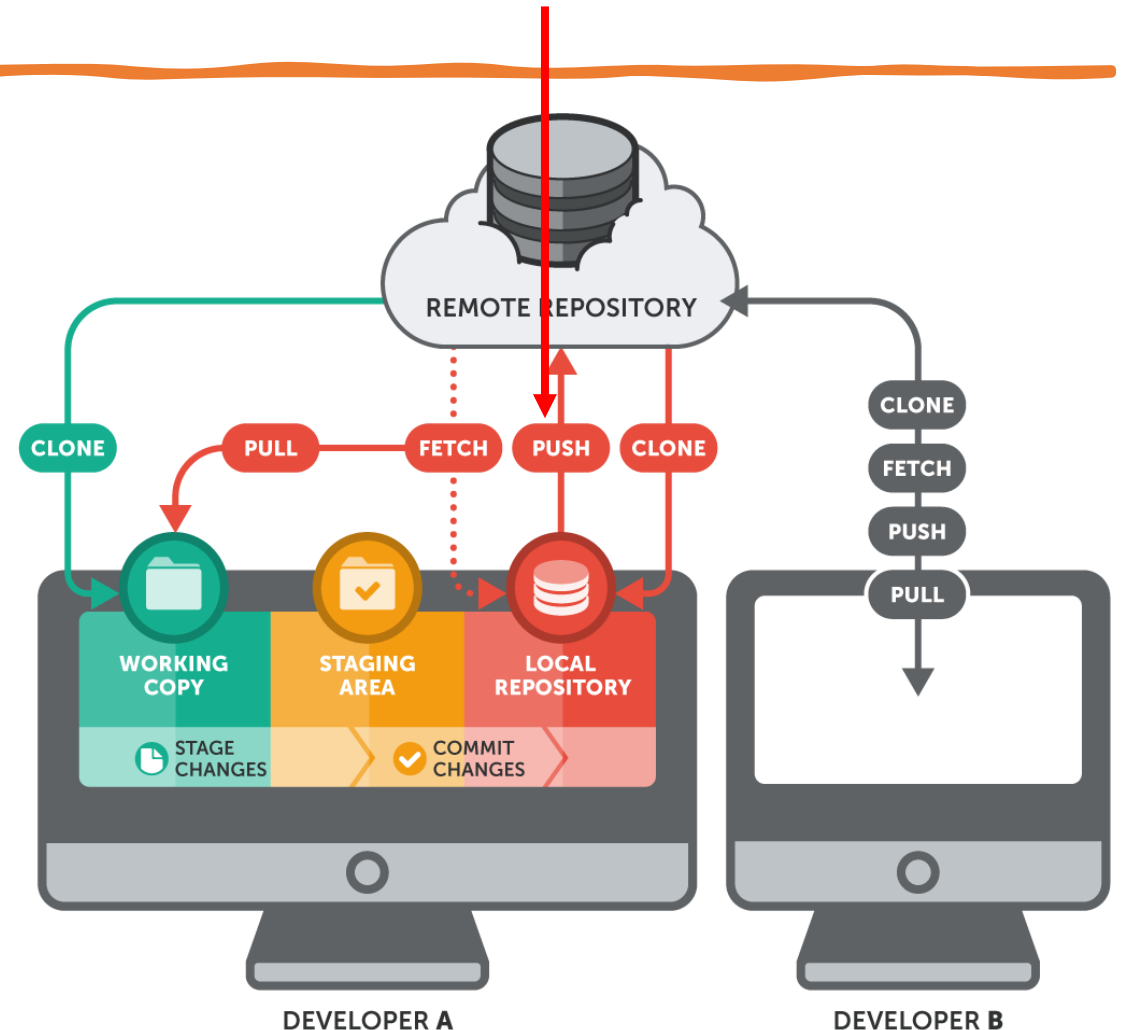
Recap of Basic Git - `git commit`

- The `git commit` command takes the current state of the staging area (the index) and creates a permanent snapshot of those changes.
- Command usage:
 - `git commit -m "Your commit message"`: Commits the staged changes with the provided message.
 - `git commit`: Opens a text editor to write a more detailed commit message.
 - `git commit -a -m "Your commit message"`: Stages all tracked files and commits them with the provided message.



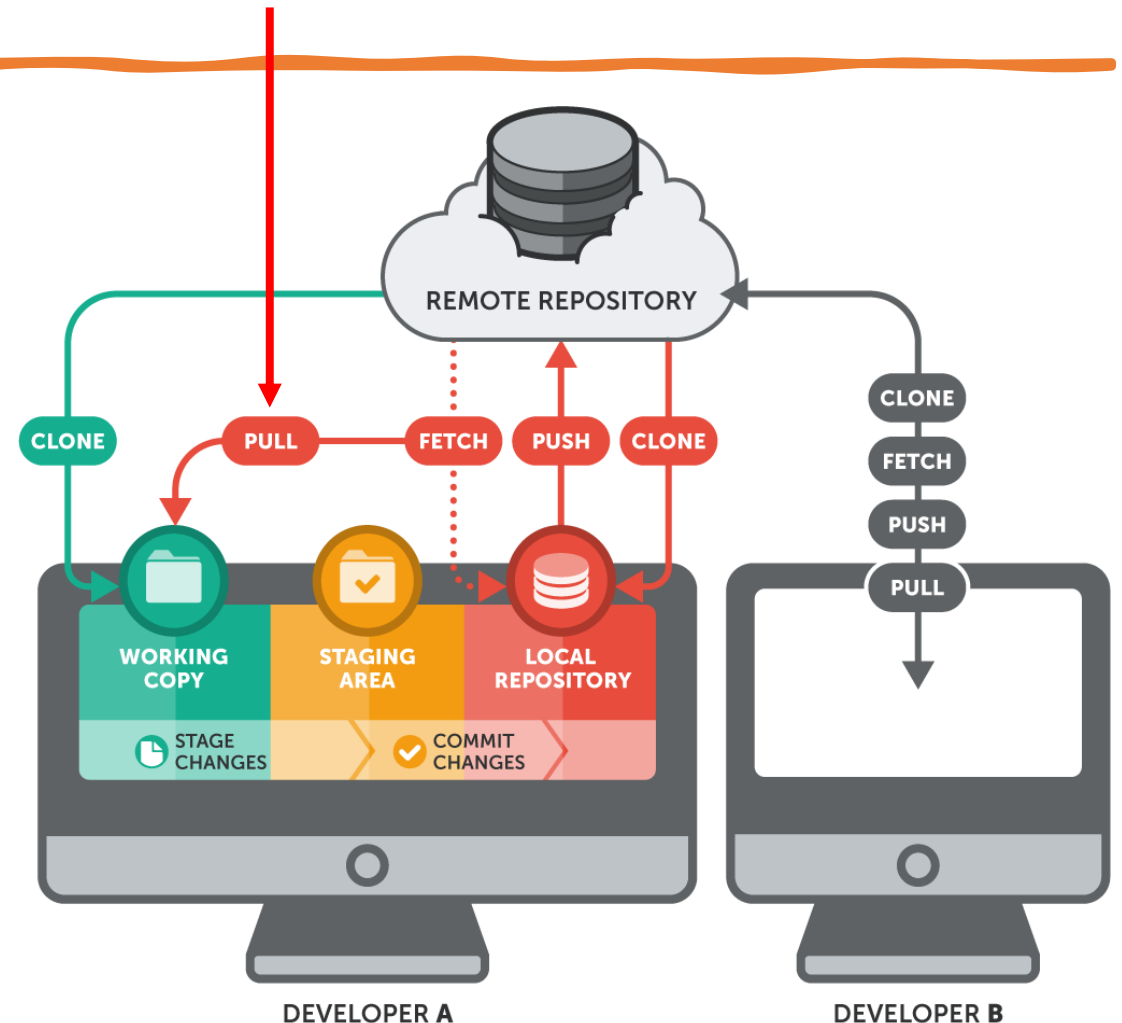
Recap of Basic Git - `git push`

- The `git push` command takes the commit you have made in your local repository and sends them to the remote repository.
- Command usage is `git push <remote> <branch>`:
 - `<remote>`: This specifies the name of the remote repository.
 - `<branch>`: This specifies the name of the local branch you want to push.



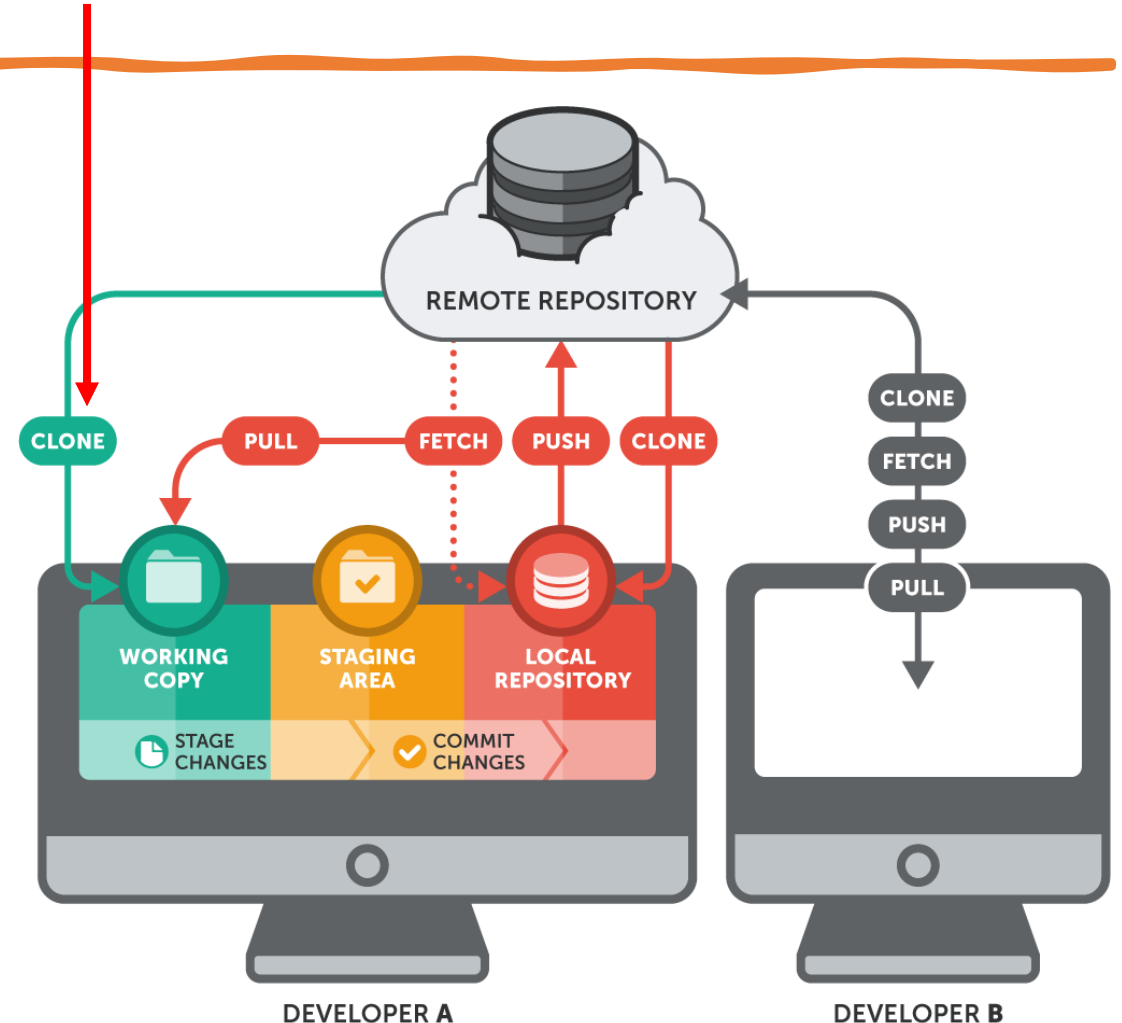
Recap of Basic Git - `git pull`

- The `git pull` command download updates from a remote repository.
- It can sometimes lead to merge conflicts if there are conflicting changes between your local branch and the remote branch.
- Difference from `git fetch`:
 - `git fetch` only downloads the changes from the remote repository; it doesn't automatically integrate them.
 - `git pull` downloads the changes and automatically integrates them.



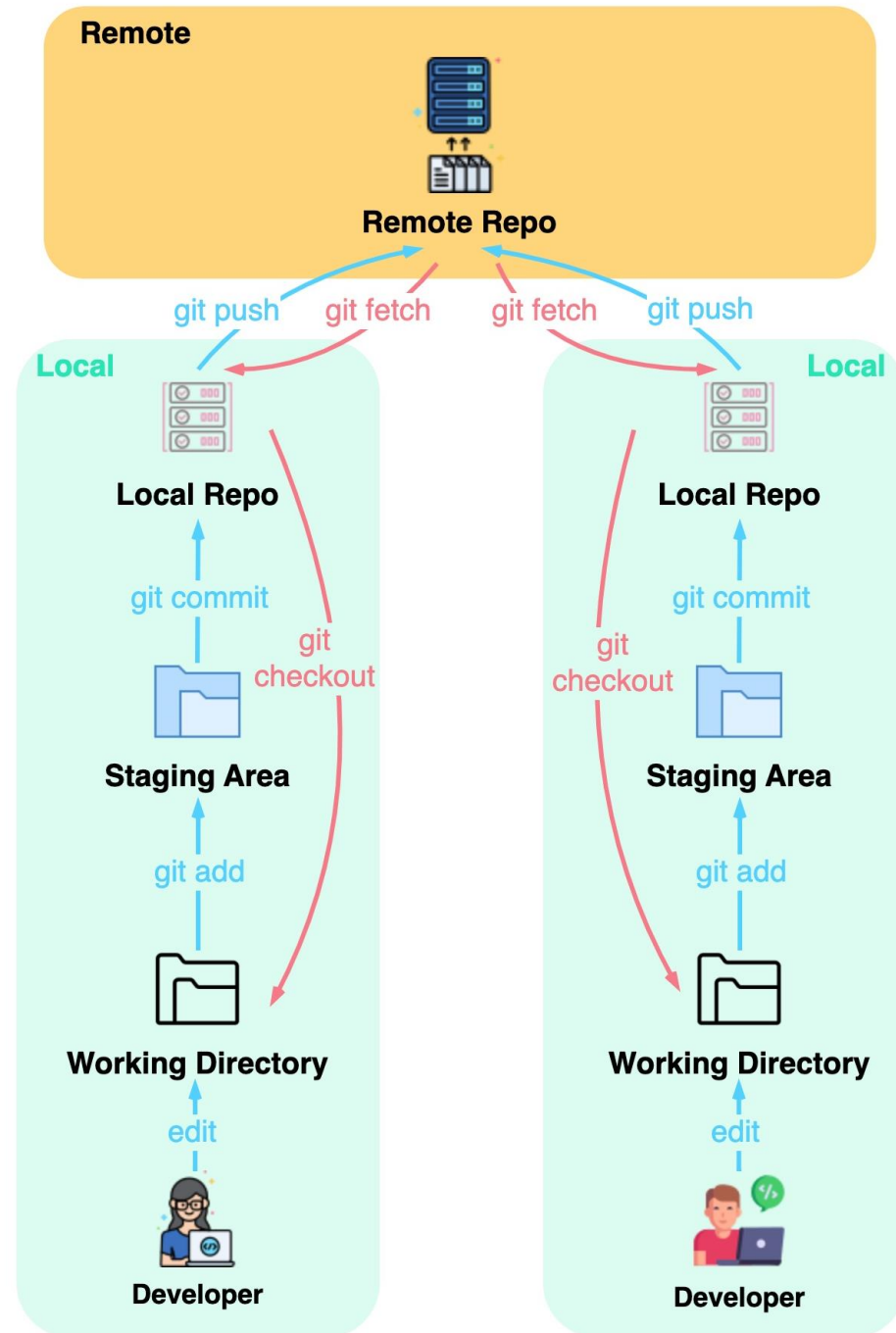
Recap of Basic Git - `git clone`

- The `git clone` command download an entire git repository from a remote source to your local machine or remote repository.
- Creates a complete copy of repository, including all files, branches and the entire commit history.
- Basic syntax is `git clone <repository-url>`:
 - `<repository-url>` is the URL of the remote git repository.



Overview

- How does git work?





Today Learning Topics

1. Introduction to Branching
2. Popular Git Workflow
3. Review of Gitflow Branching Strategy
4. Rewriting History
5. Advanced Git Commands

Introduction to Branching

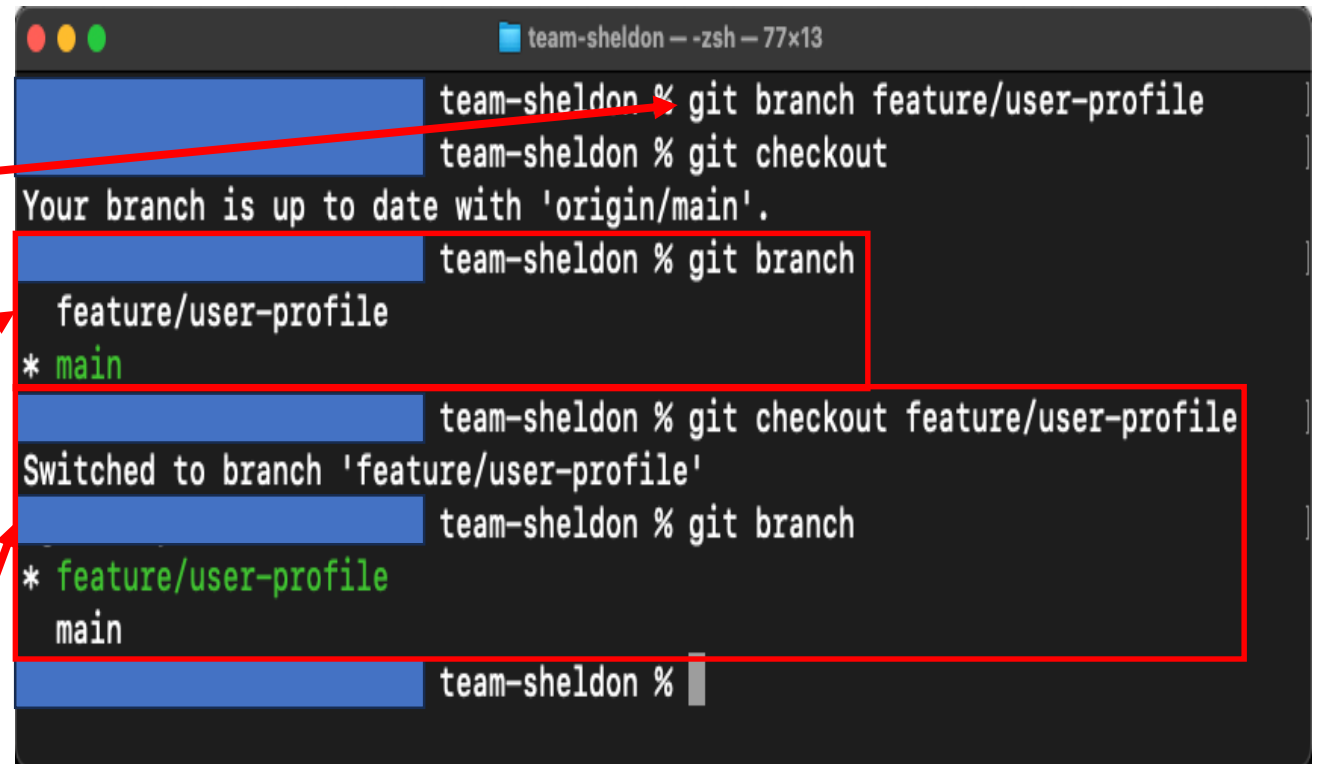
- In Git, a branch is essentially a lightweight, movable pointer to a commit.
- A branch is **NOT a copy of files or directories**, rather it is a reference to a specific commit in your repository history.
- Branches allow you to **work on different features, bug fixes, or experiments without affecting the main codebase**.
 - This enables **parallel development**, where multiple developers or teams can work on different aspects of the project simultaneously.

Introduction to Branching

- Branches provide isolation, so **changes made in one branch do not affect other branches** until they are explicitly merged.
 - This helps prevent unstable or incomplete code from being integrated into the main codebase.
- As you make commits on a branch, the branch pointer moves forward, creating a separate history of changes.
 - This history is independent of other branches until they are merged.
- When you create a new Git repository, a default branch is created, typically named "main" or "master".
 - This "main" branch represents the stable, production-ready version of your project.

Introduction to Branching

- Creating a local branch in your local repository, let's say "user-profile", using `git branch <branch-name>`:
 - E.g. `git branch feature/user-profile`
- You can verify the branches you have using the `git branch` command.
- Now, navigate to the `feature/user-profile` branch, so anything that changes only happen in that branch, using `git checkout <branch-name>` command.



```
team-sheldon % git branch feature/user-profile
team-sheldon % git checkout
Your branch is up to date with 'origin/main'.
team-sheldon % git branch
feature/user-profile
* main
team-sheldon % git checkout feature/user-profile
Switched to branch 'feature/user-profile'
team-sheldon % git branch
* feature/user-profile
main
team-sheldon %
```

The terminal screenshot shows the following sequence of commands and outputs:

- `team-sheldon % git branch feature/user-profile`
- `team-sheldon % git checkout` (Output: `Your branch is up to date with 'origin/main'.`)
- `team-sheldon % git branch` (Output: `feature/user-profile`, `* main`)
- `team-sheldon % git checkout feature/user-profile` (Output: `Switched to branch 'feature/user-profile'`)
- `team-sheldon % git branch` (Output: `* feature/user-profile`, `main`)

Red arrows from the text on the left point to the corresponding lines in the terminal: one to the first command, one to the `git branch` command and its output, and one to the `git checkout` command and its output.

Introduction to Branching

- Let say we add a new profile named “matt.md” in the contributors folder.
- Stage and commit the changes.



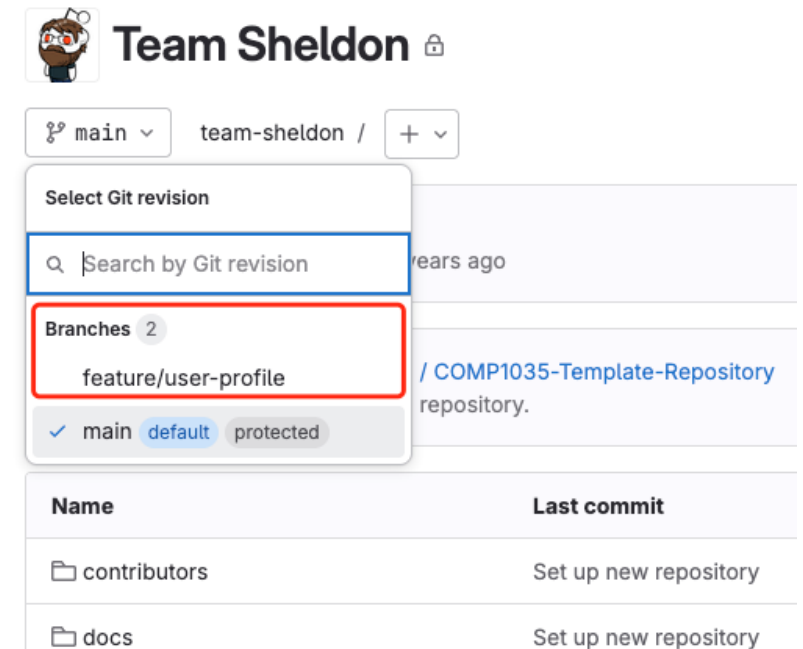
```
matt.md x
team-sheldon > contributors > matt.md > ## Matthew Pike
1  ## Matthew Pike
2
3  ![UNNC Profile](https://research.nottingham.edu.cn/en/persons/
   matthew-pike "Matthew Pike")
4
5  Dr Matthew Pike
6  Assistant Professor in Computer Science
7  School of Computer Science
8  Sensor, Sensor Networks, and Instrumentation Research Group
9
10 Matthew's primary research area is in the field of HCI.
   Specifically, he investigates the concept of Digital Wellbeing -
   the capacity of user's to look after their personal health,
   safety, relationships and work-life balance in digital settings.
   Matthew is particularly interested in the brain, and how it
   responds to our interactions with user interfaces. His PhD
   explored the direct application of psycho-physiological sensors
   for facilitating natural interactions.
11
12
```

```
team-sheldon % git add contributors/matt.md
team-sheldon % git commit -m "Add Matthew Pike user profile feature"
[feature/user-profile 3403867] Add Matthew Pike user profile feature
1 file changed, 1 deletion(-)
team-sheldon %
```

Introduction to Branching

- Now, push the `feature/user-profile` branch to the remote repository.
 - If this is the first time you push the branch, use this command:
 - `git push --set-upstream origin feature/user-profile`
 - Else, use this command:
 - `git push origin feature/user-profile`

```
% git push origin feature/user-profile
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 16 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (11/11), 1.55 KiB | 1.55 MiB/s, done.
Total 11 (delta 5), reused 0 (delta 0), pack-reused 0
remote:
remote: To create a merge request for feature/user-profile, visit:
remote:   https://csprojects.nottingham.edu.cn/[redacted]/team-sheldon/-/merge_
remote: requests/new?merge_request%5Bsource_branch%5D=feature%2Fuser-profile
remote:
remote: To https://csprojects.nottingham.edu.cn/[redacted]/team-sheldon.git
* [new branch]      feature/user-profile -> feature/user-profile
[redacted] team-sheldon %
```



Team Sheldon

main team-sheldon / +

Select Git revision

Search by Git revision

Branches 2

feature/user-profile

main default protected

Name	Last commit
contributors	Set up new repository
docs	Set up new repository

Introduction to Branching

 main team-sheldon / contributors / + ▼



Set up new repository

BOON GIIN LEE authored 2 years ago



Forked from [COMP1035_FSE / COMP1035-Template-Repository](#)

Up to date with the upstream repository.

Name	Last commit
..	
 .gitkeep	Set up new repository
 bryanlbg.md	Set up new repository

 feature/user-pro... team-sheldon / contributors / + ▼



Add Matthew Pike user profile feature

BOON GIIN LEE authored 13 minutes ago



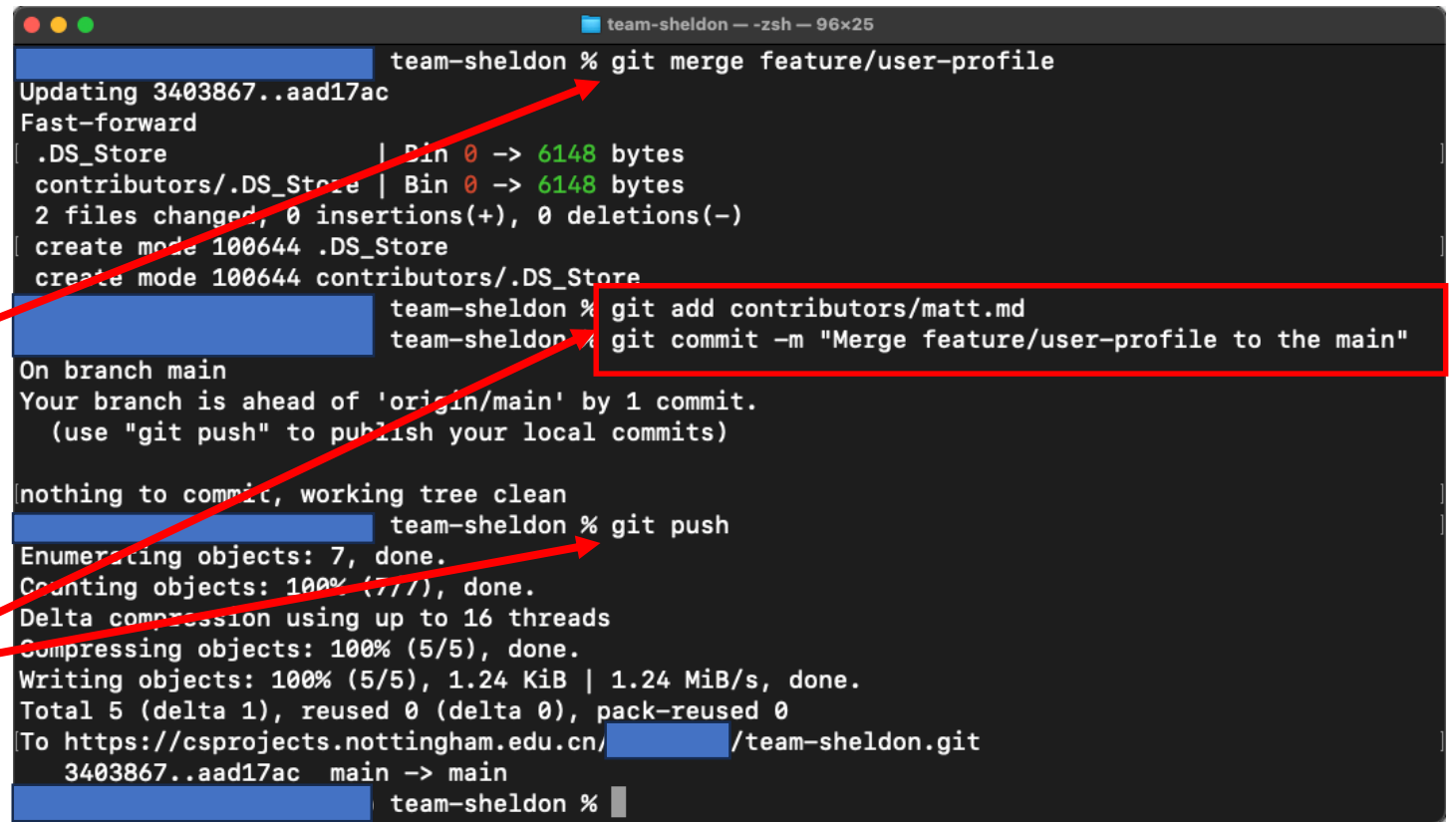
Forked from [COMP1035_FSE / COMP1035-Template-Repository](#)

5 commits ahead of the upstream repository.

Name	Last commit
..	
 .gitkeep	Set up new repository
 bryanlbg.md	Set up new repository
 matt.md	Add Matthew Pike user profile feature

Introduction to Branching

- If you confirm that about the change and everything is reviewed and tested (for coding), you may now consider to merge it into the main branch.
- First, switch back to the main branch, then use the `git merge <branch-name>` command to merge the `<branch-name>` branch into the main.
- Then, stage and commit the merged changes and push the changes to the remote repository.



```
team-sheldon % git merge feature/user-profile
Updating 3403867..aad17ac
Fast-forward
 .DS_Store | Bin 0 -> 6148 bytes
 contributors/.DS_Store | Bin 0 -> 6148 bytes
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 .DS_Store
 create mode 100644 contributors/.DS_Store
team-sheldon % git add contributors/matt.md
team-sheldon % git commit -m "Merge feature/user-profile to the main"
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)

nothing to commit, working tree clean
team-sheldon % git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 1.24 KiB | 1.24 MiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0
To https://csprojects.nottingham.edu.cn/[redacted]/team-sheldon.git
 3403867..aad17ac main -> main
team-sheldon %
```

The terminal screenshot shows the execution of the following commands and their output:

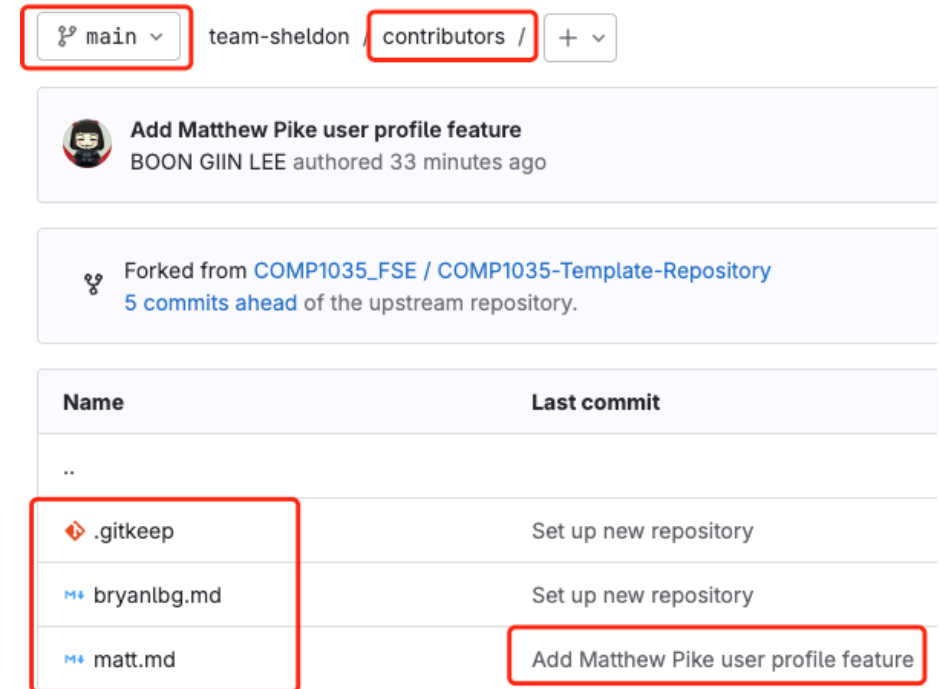
- `git merge feature/user-profile`: Merges the feature branch into main. Output shows a fast-forward merge of 2 files.
- `git add contributors/matt.md`: Stages the new file.
- `git commit -m "Merge feature/user-profile to the main"`: Commits the merge.
- `git push`: Pushes the commit to the remote repository.

Red arrows from the text on the left point to the corresponding commands in the terminal:

- From "use the `git merge <branch-name>` command" to `git merge feature/user-profile`.
- From "use the `git add` command" to `git add contributors/matt.md`.
- From "use the `git commit` command" to `git commit -m "Merge feature/user-profile to the main"`.
- From "push the changes to the remote repository" to `git push`.

Introduction to Branching

- `git branch feature/user-profile`: Creates a new branch named `feature/user-profile`.
- `git checkout feature/user-profile`: Creates and switches to the new branch in one step.
- `git add contributors/matt.md`: Stages the changes in the `matt.md` file in the `contributors` folder.
- `git commit -m "Add Matthew Pike user profile feature"`: Commits the staged changes with a descriptive message.
- `git push origin feature/user-profile`: Pushes the local branch to the remote repository.
- `git branch -d feature/user-profile`: Deletes the `feature/user-profile` local branch.
- `git push origin feature/user-profile`: Push the `feature/user-profile` branch to the remote repository.



The screenshot shows a GitHub repository interface. At the top, the repository name is 'team-sheldon/contributors'. Below this, a commit by 'BOON GIIN LEE' is shown with the message 'Add Matthew Pike user profile feature'. The commit is 33 minutes old. Below the commit, it says 'Forked from COMP1035_FSE / COMP1035-Template-Repository' with '5 commits ahead' of the upstream repository. A table below shows the file list:

Name	Last commit
..	
.gitkeep	Set up new repository
bryanlbg.md	Set up new repository
matt.md	Add Matthew Pike user profile feature

Advanced Git Workflow

- A Git workflow is a recommended practice or a set of rules that teams use to manage their Git repositories and collaborate effectively.
- It defines how changes are introduced, reviewed, and integrated into the main codebase.
- Choosing the **right Git workflow** is crucial for maintaining a clean and organized project history, especially in collaborative environments. It depends on several **factors**:
 - Team size and experience.
 - Project complexity.
 - Release frequency.
 - Deployment strategy.

Advanced Git Workflow

Strategy	Core Principle	Ideal Team Size	Release Cadence	Best Suited For
Gitflow	Strict branching model around releases	Larger teams	Scheduled releases	Projects with formalized release processes and long-term support needs
GitHub Flow	Anything in main is deployable	Smaller teams	Frequent releases, continuous delivery	Web applications, projects with rapid iteration
GitLab Flow	Feature branching with issue tracking and environment branches	Medium to large teams	Flexible, versioned or continuous releases	Projects needing multiple environments and structured release management
Trunk-Based Development	Integrate to main daily	Smaller to medium teams	Continuous integration and delivery	Projects prioritizing speed, frequent updates, and high collaboration

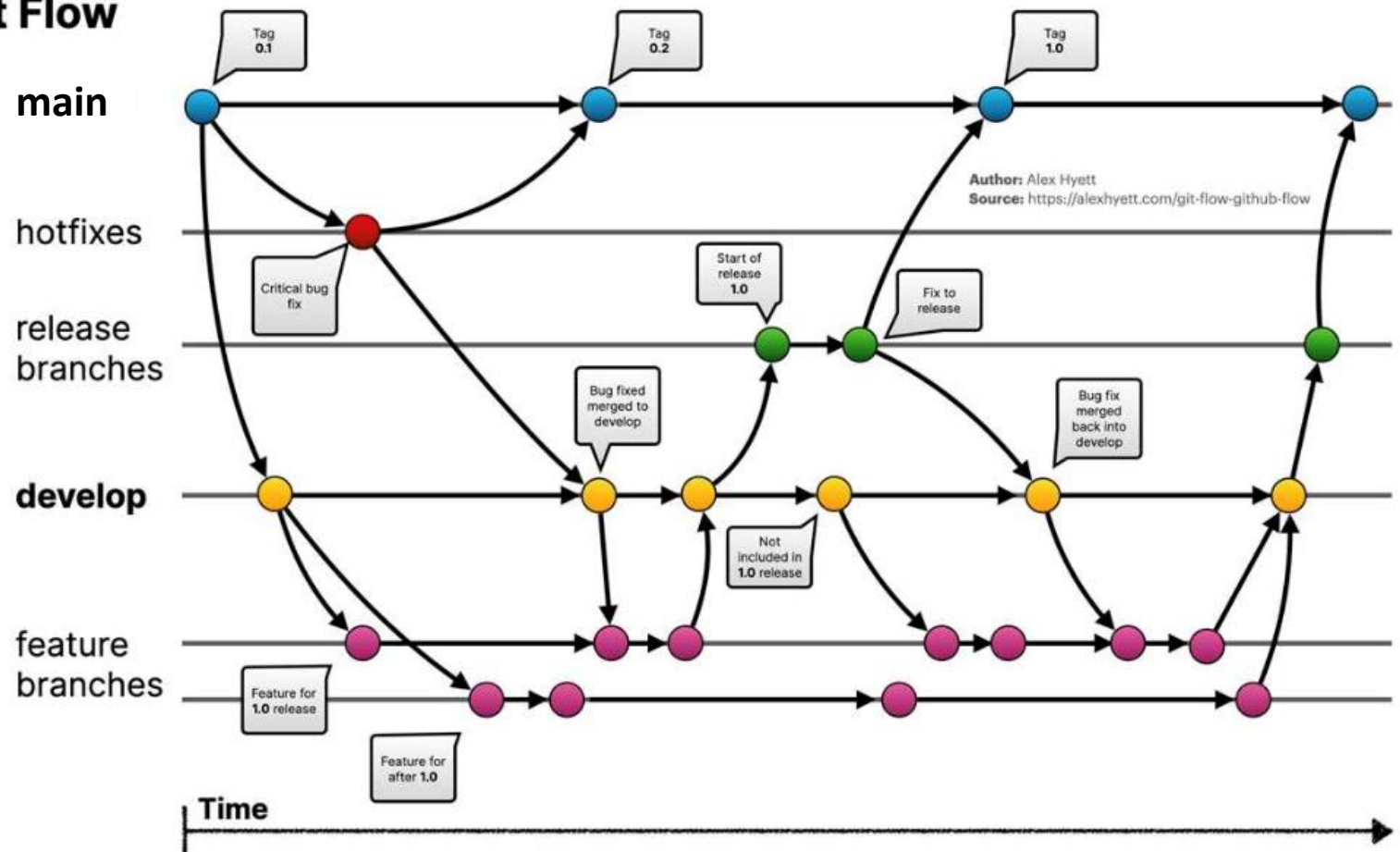
Gitflow Strategy

- The Gitflow workflow defines a **strict branching model** centred around project releases, providing a robust framework particularly suitable for managing large projects .
- This model employs several core branches with specific roles.
- Best **suited** for:
 - Projects with predictable release schedules.
 - Larger development teams.
 - Need for a formalised approach to branching and release management.
- **Considerations:**
 - Can be complex.
 - Longer branch lifespans
 - Potential challenges with Continuous Integration and Continuous Delivery (CI/CD) practices.

Gitflow Strategy

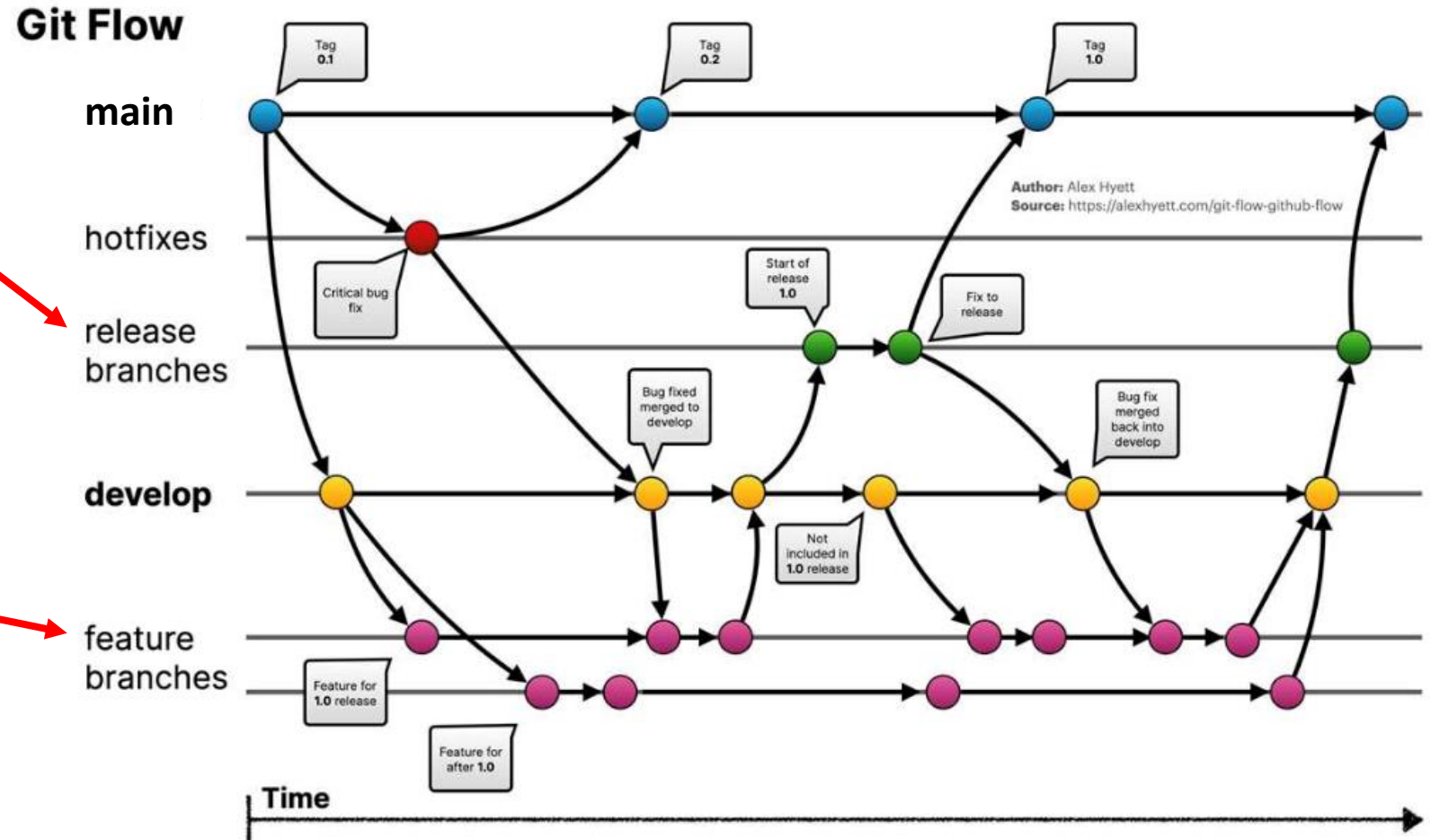
- `main/master`:
 - Stores the official release history.
 - Each commit ideally represents a tagged release.
- `develop`:
 - Integration branch for all features.
 - Holds the complete history of ongoing development.

Git Flow



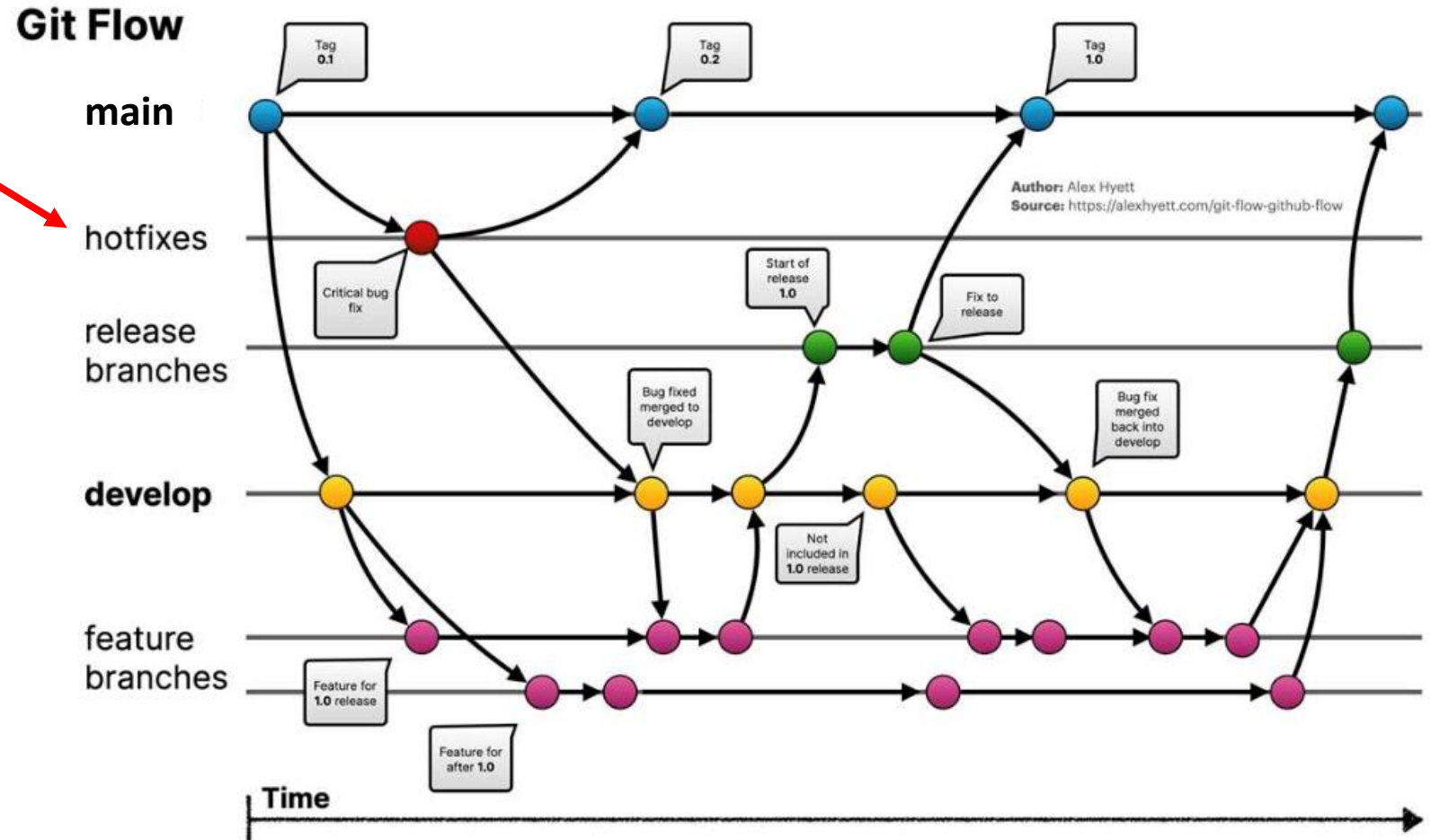
Advanced Git Workflow

- release:
 - Used for release preparation.
 - Allows for final bug fixes and stabilisation.
 - Merged into both main and develop.
- feature:
 - Created for developing new features.



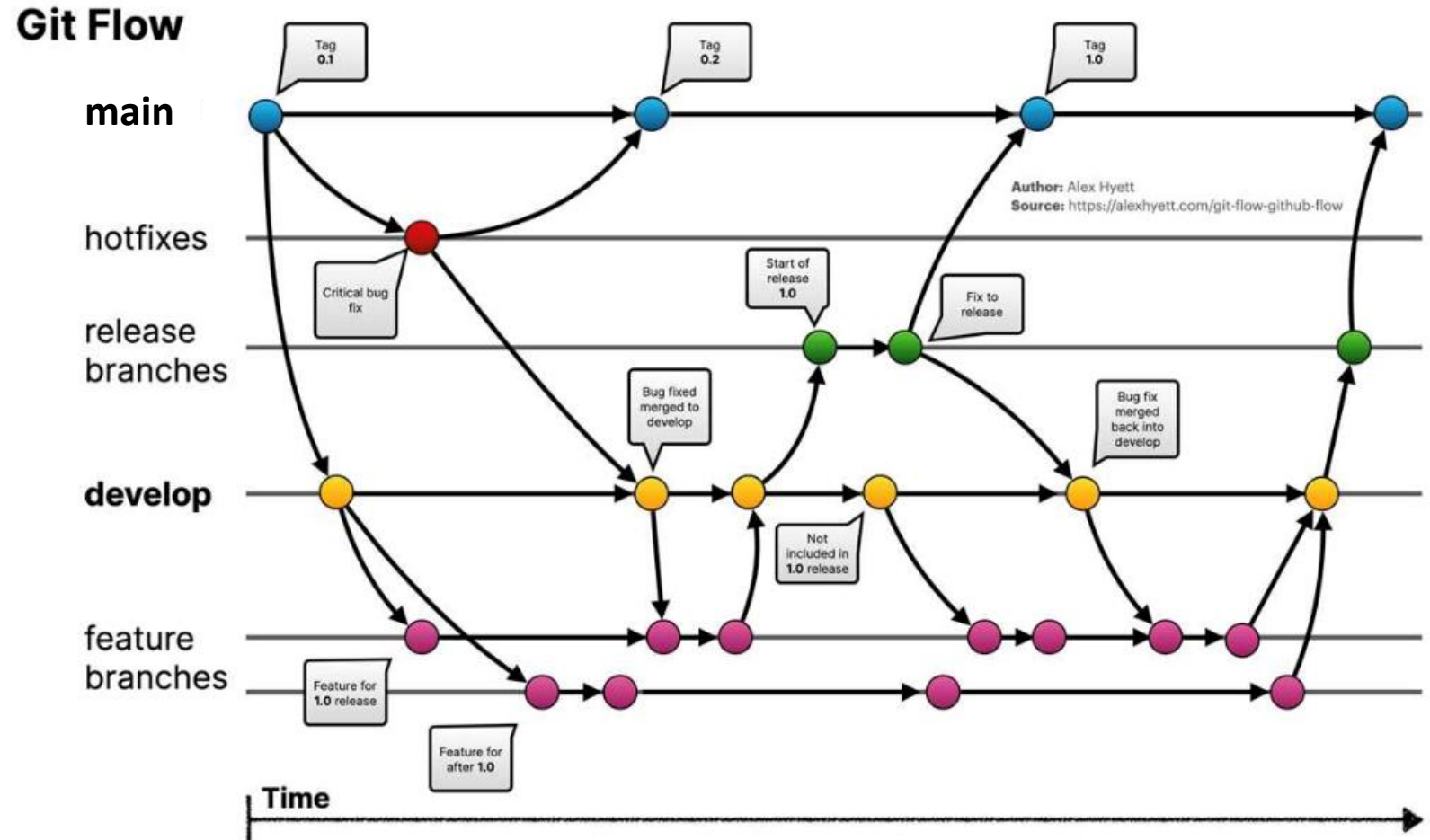
Gitflow Strategy

- hotfix/bug:
 - Used for critical fixes to production released.
 - Addresses urgent issues without disrupting ongoing development.



Gitflow Strategy

- **Workflow Process:**
 - feature branches from develop.
 - Merge feature branches back to develop.
 - release branches for stabilisation.
 - hotfix branches from main for urgent fixes.



Gitflow Branching Naming Convention

Element	develop	feature	release	hotfix
Prefix	develop/	feature/	release/	hotfix/
Example	develop/1.0	feature/user-authentication feature/payment-integration	release/1.2 release/2.0-rc1	hotfix/1.2.1 hotfix/security-patch
Purpose	Used for the development pipeline	Used for developing new features	Used for preparing a new release	Used for addressing critical bugs in production releases
Origin	Branch off from main	Branch off from develop	Branch off from develop	Branch off from main
Merge Target	Merge into release	Merge back into develop	Merge into both main and develop	Merge into both main and develop (or develop only, depending on the need)



Rewriting History

git rebase

- The `git rebase` command that integrates changes from one branch into another, similar to `git merge`.
- However, instead of creating a merge commit, `git rebase` rewrites the commit history by moving or combining commits.
- This process effectively rewrites the commit history, making it appear as if the feature branch was created directly from the latest commit of the target branch.

git rebase

- Let say we have a `main` branch and a `feature/user-profile` branch.

- **Initial State:**

```
A -> B -> C (main)
      |
      v
      D -> E (feature/user-profile)
```

- **Rebasing** `feature/user-profile` into `main`:

- `git checkout feature/user-profile`
- `git rebase main`

- **Result:**

```
A -> B -> C (main)
      |
      v
      D' -> E' (feature/new-feature)
```

git rebase

- When **to Use** Git Rebase:
 - When you want a clean, linear history.
 - When working on a feature branch that hasn't been shared yet.
 - When you want to integrate changes from main into your feature branch before creating a pull request.
- When **to Avoid** Git Rebase:
 - When working on a branch that has already been pushed to a shared repository.
 - When you want to preserve the exact commit history.
 - When working in a team where rebasing is not the standard practice.

rebase vs merge

Feature	Git Merge	Git Rebase
Mechanism	Creates a new "merge commit" that joins branch histories.	Replays commits from the source branch onto the target branch.
Commit History	Preserves the entire history of both branches, including the merge commit.	Creates a linear commit history by rewriting commit history.
Merge Commits	Introduces a merge commit.	Avoids merge commits.
History Appearance	Shows when and how the integration occurred, reflecting the actual branching and merging.	Appears as if all changes were made sequentially on the target branch.
Complexity	Generally simpler to understand and use.	Can be more complex, especially when resolving conflicts.
Potential Conflicts	Fewer conflicts during the merge itself, but the merge commit may be complex.	Can lead to more frequent and complex conflicts during the rebase process.
Impact on Shared Branches	Safe for shared branches, as it doesn't rewrite history.	Potentially problematic for shared branches, as it rewrites history.
History Clarity	Reflects the branching and merging process, which can be complex.	Creates a clean, linear narrative of development, easier to follow.
Team Preference	Often preferred when historical accuracy is paramount.	Often preferred when a clean and linear history is desired.

git reset

- This is powerful command in Git that is used to move the current branch HEAD to a specified commit.
- It is important to understand that it can also affect the staging area (index) and the working directory, depending on the options used.
- Command usage of `git reset`:
 - `--soft`:
 - `--mixed` (default)
 - `--hard`

git reset

- `git reset --soft <commit>`:
 - Scenario: You made a commit with the wrong message.
 - Example:
 1. `git commit -m "Wrong commit message"`
 2. `git reset --soft HEAD~1` **# Move HEAD back one commit**
 3. `git commit -m "Correct commit message"` **# Re-commit with the correct message**
 - Explanation: This moves the HEAD back to the previous commit, but your changes are still staged. You can then re-commit with the correct message.
 - You can use `git log` to check the commit status (type `q` in the ":" to quit).

git reset

- `git reset --mixed <commit>`:
 - Scenario: You made a commit and now want to undo the commit and unstage the changes.
 - Example:
 1. `git add contributors/matt.md`
 2. `git commit -m "Made some changes"`
 3. `git reset --mixed HEAD~1 # Move HEAD back one commit and unstage changes.`
 - Explanation: This moves the HEAD back to the previous commit and unstages the changes. The changes are still present in your working directory, but you can now modify them and stage them again.

git reset

- `git reset --hard <commit>`:
 - Scenario: You made a series of commits that you want to completely discard.
 - Example:
 1. `git add contributors/matt.md`
 2. `git commit -m "Made some changes"`
 3. `git reset --hard HEAD~1` # Move HEAD back one commit and discard changes.
 - Explanation: This moves the HEAD back to the previous commit and discards all changes made since that commit. **Use this with extreme caution, as it results in data loss.**

git reset

- `HEAD^` refers to the commit before the current HEAD. You can also use commit SHAs (e.g., `git reset --hard abc123def`).
- `git reset --hard` can result in data loss, so make sure you understand the implications before using it.
- Avoid using `git reset` on public branches that others are working on, as it can rewrite history and cause conflicts.
- Always be aware of the state of your working directory and staging area before using `git reset`.
- If you accidentally hard reset, there is a chance to recover the lost commits by using the command `git reflog`.

Summary

- Define a Git repository, commit, branch, staging area, and remote repository.
- Describe common Git workflows such as Gitflow (feature/hotfix/release/develop/main branches).
- Understand and utilise the Gitflow branching naming conventions.
- Create, switch, merge, and rebase branches effectively.
- Understand the implications of rewriting commit history with `git rebase`.
- Explored advanced Git commands such as `git merge` and `git reset`.



THANK

YOU